

Problem 1 - Washing Machine Mount Design

The Matlab code for this problem is listed in Appendix 1.

Problem 1a

Equation calculates the force produced by the imbalanced 1kg mass at a 0.5 m radius from the center of the washing machine as a function of angular velocity,

$$F = m \cdot \omega^2 \cdot r \quad (1)$$

where m is 0.5 kg, ω is the angular velocity in $\frac{\text{radians}}{\text{s}}$, and r is 0.5 m.

Figure 1 shows the force produced by the 1 kg load as a function of angular velocity in RPM.

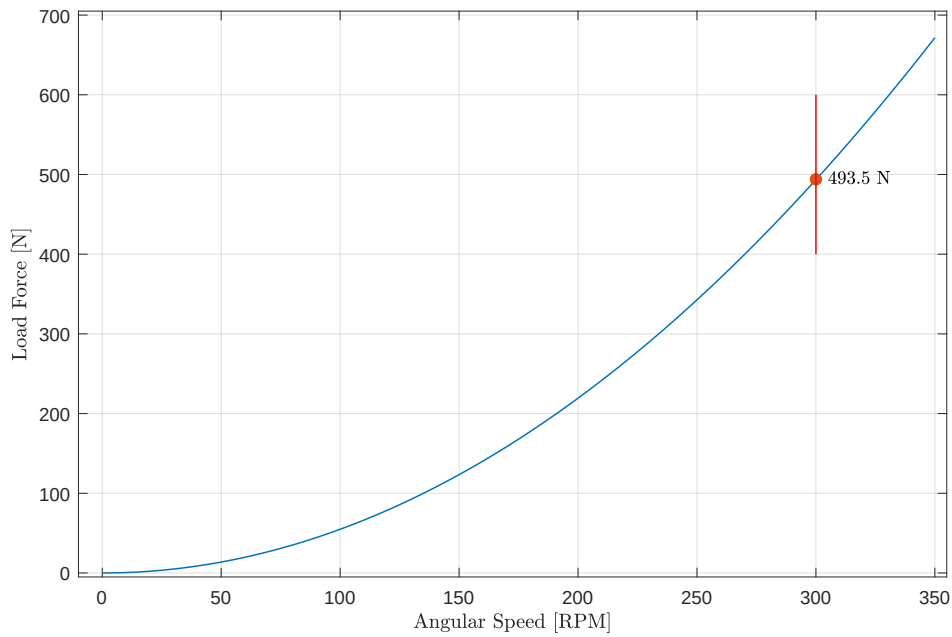


Figure 1: Load force versus washing machine angular speed.

At 300 RPM, or 5 Hz, the imbalanced load force amplitude is 493.5 N.

Problem 1b

The required spring stiffness, k_s , is $9,810 \frac{\text{N}}{\text{m}}$. The mount's natural frequency, ω_o , is 1.5 Hz.

Problem 1c

Figure 2 shows the transmission loss of the system for a set of damping ratios ranging from 0.001 to 1.

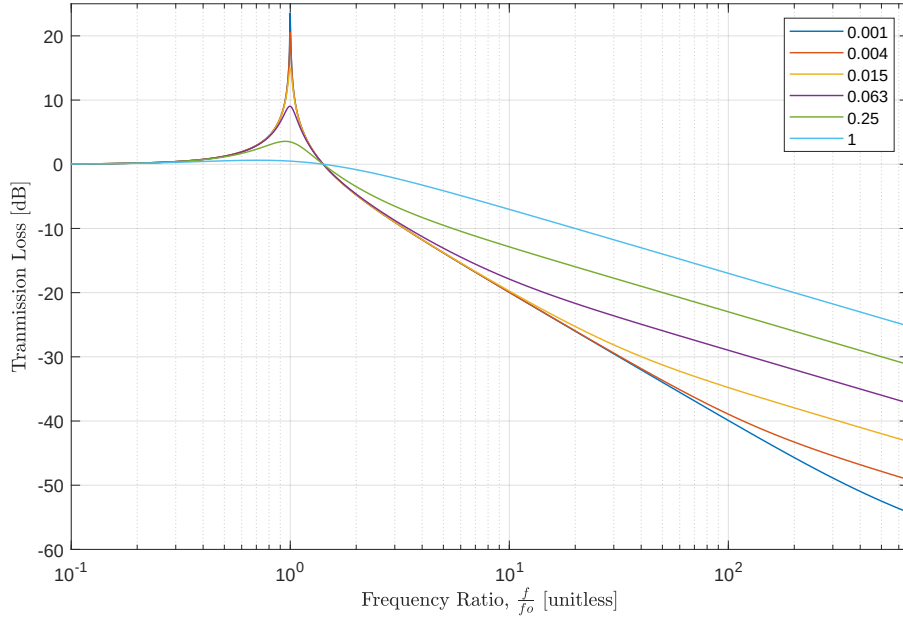


Figure 2: Transmission loss as a function of damping ratio. The natural frequency is 1.5 Hz.

Problem 1d

Figure 3 shows the force applied to the foundation.

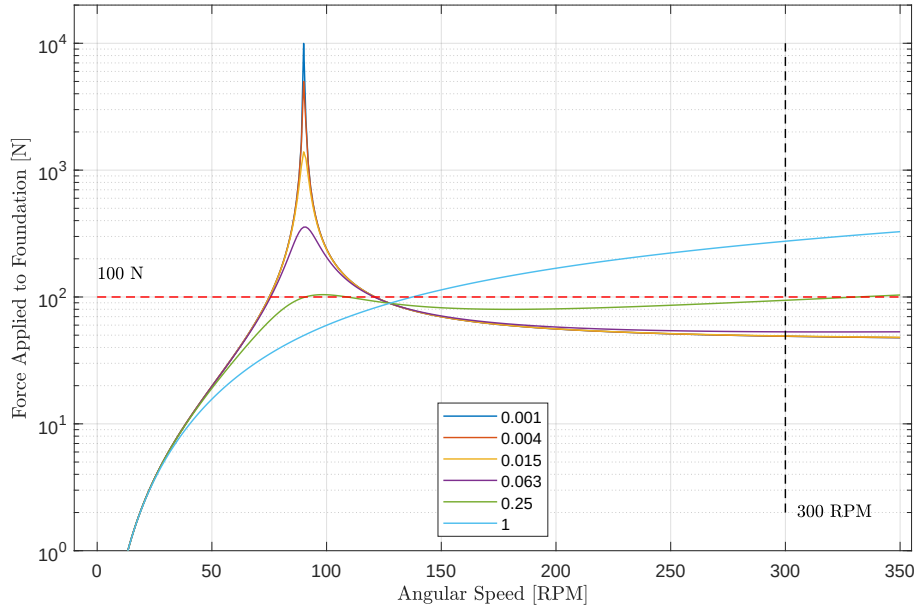


Figure 3: Force applied to the foundation as a function of damping ratio.

No damping ratio makes the applied force less than 100 N across the 0 to 350 RPM operating range.

The best damping ratio is $\zeta = 0.25$. The force applied to the foundation still exceeds the 100 N target in only two small RPM regions: 91 to 109, and 331 to 350 RPM. In both of these regions the force is approximately 104 N.

Problem 1e

The viscous damping coefficient is $519.4 \frac{\text{kg}}{\text{s}}$ based on a total mass of 110 kg.

Figure 4 shows the admittance.

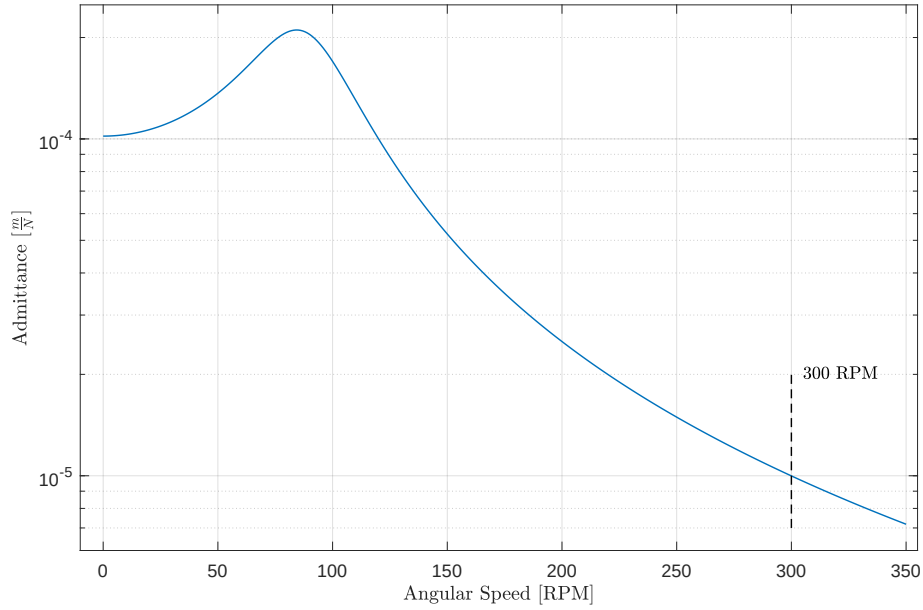


Figure 4: Admittance versus RPM.

The admittance at 300 RPM is $10^{-5} \frac{\text{m}}{\text{N}}$.

Problem 1f

Figure 5 shows the displacement of the washing machine.

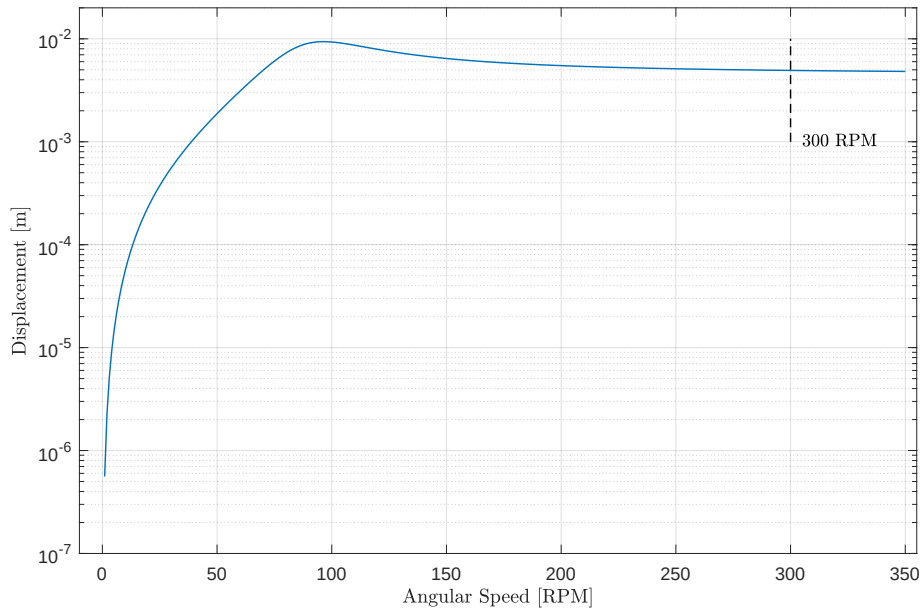


Figure 5: Displacement versus RPM.

The displacement of the washing machine at 300 RPM is approximately 4.9×10^{-3} m. The washing machine undergoes larger momentary displacements as it spins up to 300 RPM, in particular around 100 RPM.

Problem 2 - Washing Machine Mount Design (2 DOF)

The Matlab code for this problem is listed in Appendix 2.

Problem 2a

Figure 1 illustrates the two-stage vibration mount showing the washing machine and the “raft”.

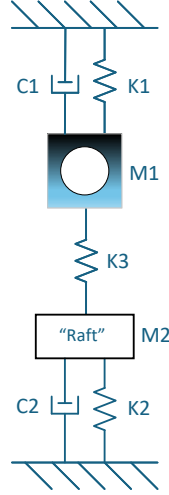


Figure 1: Two-stage vibration mount system for the washing machine.

Problem 2b

Table 1 lists the initial stiffness and damping coefficients for the system.

M1	110	kg
K1	100	$\frac{\text{N}}{\text{m}}$
C1	5	$\frac{\text{kg}}{\text{s}}$ ¹

M2	100	kg
K2	100	$\frac{\text{N}}{\text{m}}$
C2	5	$\frac{\text{kg}}{\text{s}}$ ²

M3	0	kg
K3	1,000	$\frac{\text{N}}{\text{m}}$
C3	0	$\frac{\text{kg}}{\text{s}}$ ³

Table 1: Initial values of stiffness and damping coefficients.

Damping was implemented using the damping vector⁴.

¹Element of damping vector.

²Element of damping vector.

³Element of damping vector.

⁴Not equivalent to using a complex-valued stiffness vector; the imaginary part is the damping for the respective spring

Problem 2c

Figure 2 shows the admittance of the washing machine and the raft.

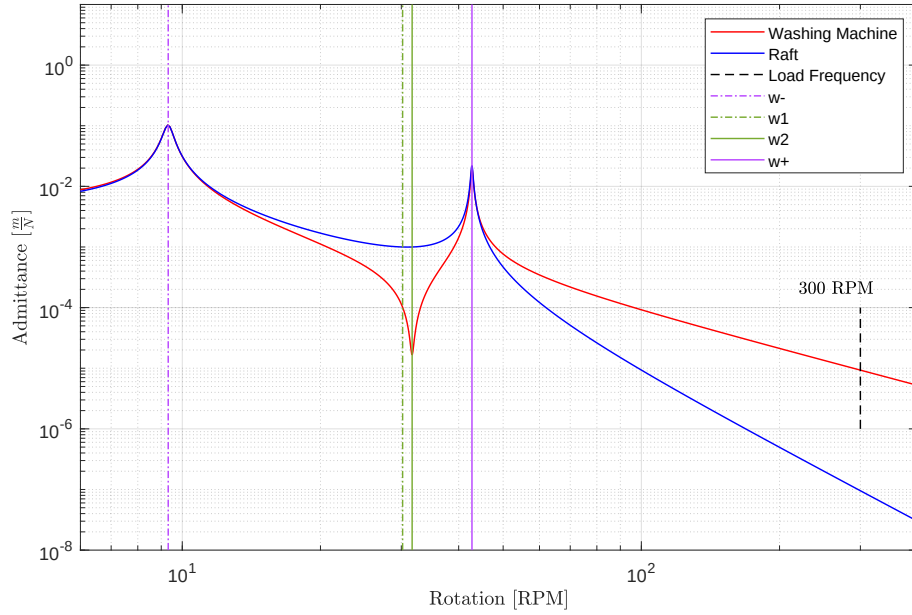


Figure 2: Admittance of the washing machine and the raft with a force on the washing machine.

The admittance of the washing machine is $9.31 \times 10^{-6} \frac{\text{m}}{\text{N}}$ at 300 RPM.

Problem 2d

Figure 3 show the displacement of the washing machine and the raft.

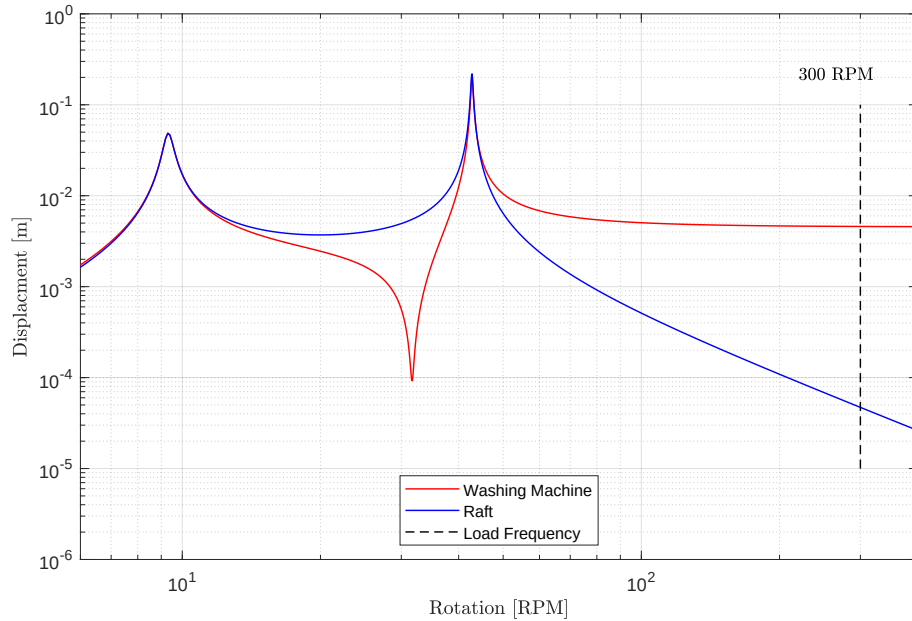


Figure 3: Displacement of the washing machine and the raft with a force on the washing machine.

At 300 RPM, the washing machine displacement is $4.6 \times 10^{-3} \text{ m}$, less than $5.0 \times 10^{-3} \text{ m}$ for the first-order system. The displacement of the raft is $4.7 \times 10^{-5} \text{ m}$.

Problem 2e

Figure 4 shows the force applied to the ground by the raft.

The force is calculated by multiplying the displacement of the raft by the K_2 spring constant.

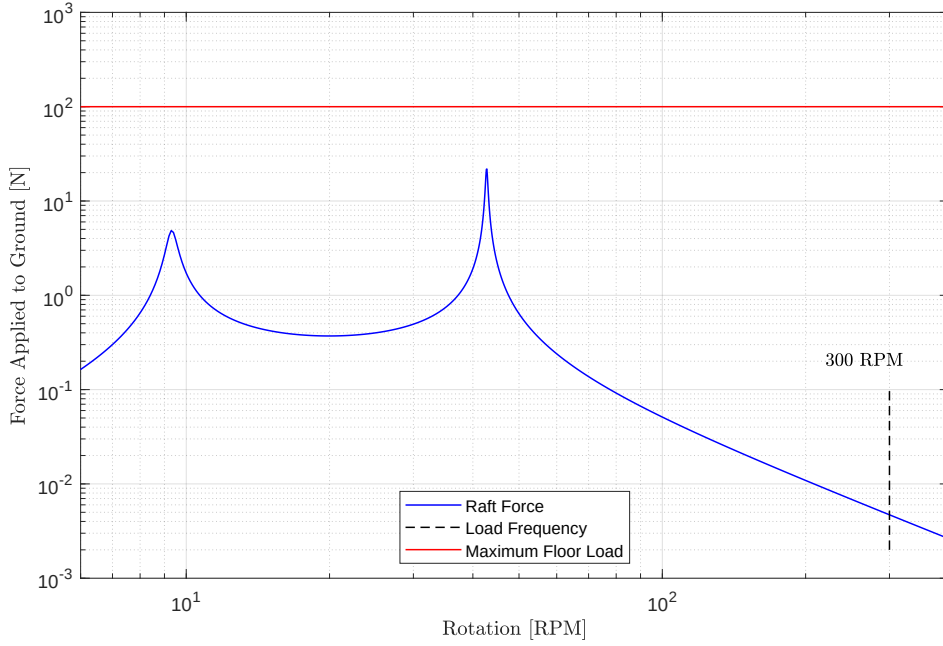


Figure 4: Force applied to the ground by the raft.

The ground force at 300 RPM is 4.7×10^{-3} N. The force on the ground is always less than 100 N.

Problem 2f

The performance of this system was improved by *lowering the stiffness of the coupling spring*, k_3 . Lowering k_3 decreases the separation of the two resonance peaks and shifts them down in frequency. This results in lower admittances for the washing machine and the raft at 300 RPM. However, due to the quadratic characteristic of the load force with RPM and the second-order roll-off of the admittance curve for the washing machine, optimization is restricted.

In Figure 3, displacement of the washing machine after the second resonance frequency “plateaus” with much less roll-off relative to the raft displacement. This restricts the degree of optimization that can be done in this case.

Problem 2g

Table 2 compares the admittance, displacement, and ground force for the initial and optimized systems.

	Admittance [$\frac{\text{m}}{\text{N}}$]	Displacement [m]	Ground Force [N]
Initial Design	9.31×10^{-3}	4.59×10^{-3}	4.7×10^{-3}
Optimized Design	9.22×10^{-6}	4.55×10^{-3}	4.62×10^{-5}

Table 2: Performance comparison of the initial and optimized designs.

It is apparent that the optimized design greatly reduces the admittance and the ground force. However, the change in displacement is small in comparison.

Problem 3 - Washing Machine Dynamic Vibration Absorber Design

The Matlab code for this problem is listed in Appendix 3.

Part 3a

Figure 1 illustrates the dynamic vibration absorber (DVA) system.

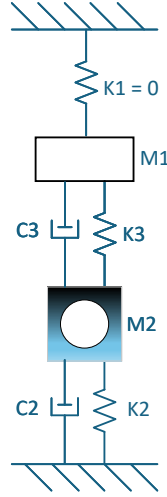


Figure 1: Dynamic vibration absorber system for the washing machine.

Table 3 lists the stiffness and damping coefficients for the system.

M1	15	kg
K1	0	$\frac{\text{N}}{\text{m}}$
C1	0	$\frac{\text{kg}}{\text{s}}$

M2	110	kg
K2	9,810	$\frac{\text{N}}{\text{m}}$
C2	1,962	$\frac{\text{kg}}{\text{s}}$ ⁵

M3	0	kg
K3	1e5	$\frac{\text{N}}{\text{m}}$
C3	1e4	$\frac{\text{kg}}{\text{s}}$ ⁶

Table 3: Stiffness and damping coefficients for the DVA system.

Damping was implemented by adding with an imaginary component to the respective spring stiffness⁷.

⁵0.2 fraction of respective spring stiffness.

⁶0.1 fraction of respective spring stiffness.

⁷Not equivalent to using a damping vector in the argument to the given Matlab nDOF function script-file.

Part 3b

Figure 2 shows the admittance response of the original system and DVA system based on the equations presented in the tutorial lecture on DVA system analysis.

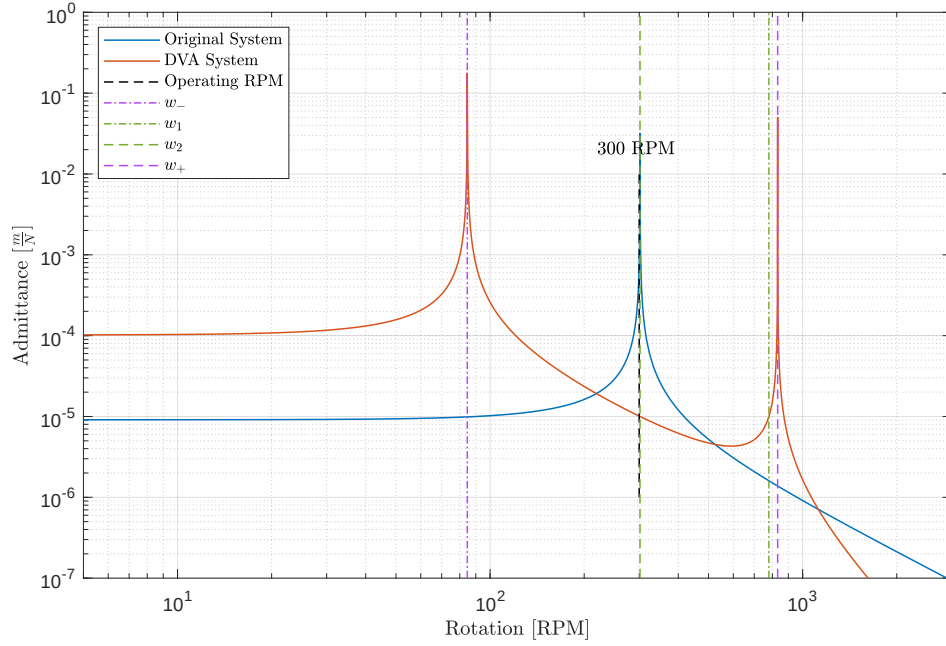


Figure 2: Admittance responses based on the equations from the DVA tutorial lecture.

The washing machine admittance at 300 RPM is reduced from the blue curve resonance peak to a point between the two resonances on the red curve).

Part 3c - Admittance

Figure 3 shows the admittance response of the DVA system using the nDOF function script-file. The stiffness of the coupling spring, k_3 , is set to place the null of the admittance response at the 300 RPM operating state of the washing machine. *The admittance at 300 RPM is approximately $1.2 \times 10^{-6} \frac{m}{N}$.*

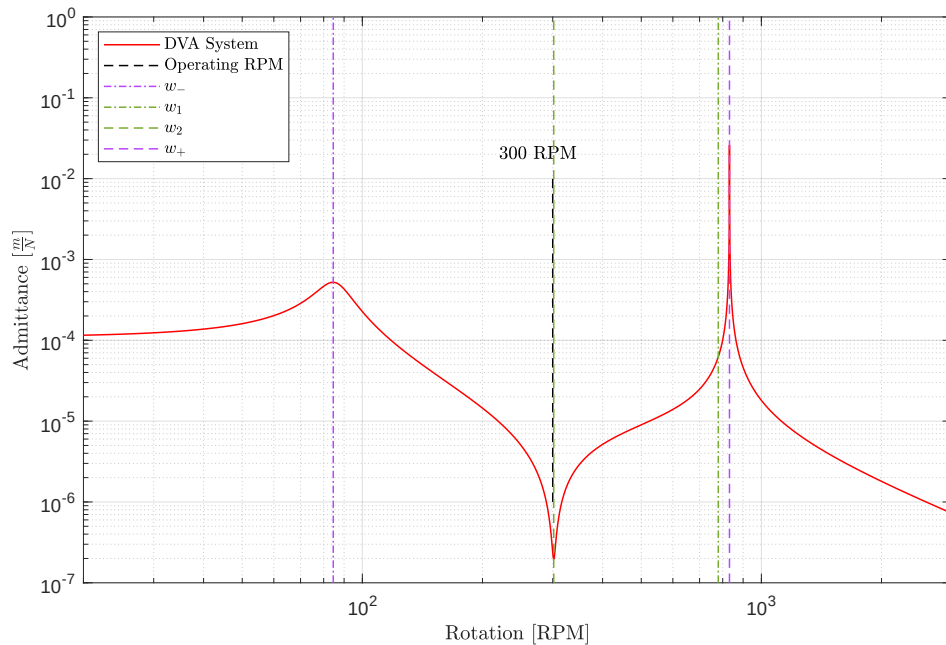


Figure 3: Solution using nDOF script-file function.

Part 3d - Displacement

Figure 4 shows the displacement of the washing machine as a function of RPM. The ramp up to 300 RPM occurs quickly so that the large displacements are transient and the maximum speed is 350 RPM (the large displacement associated with the resonance peak at higher frequencies is not reached). *The displacement at 300 RPM is approximately 38.2×10^{-6} m.*

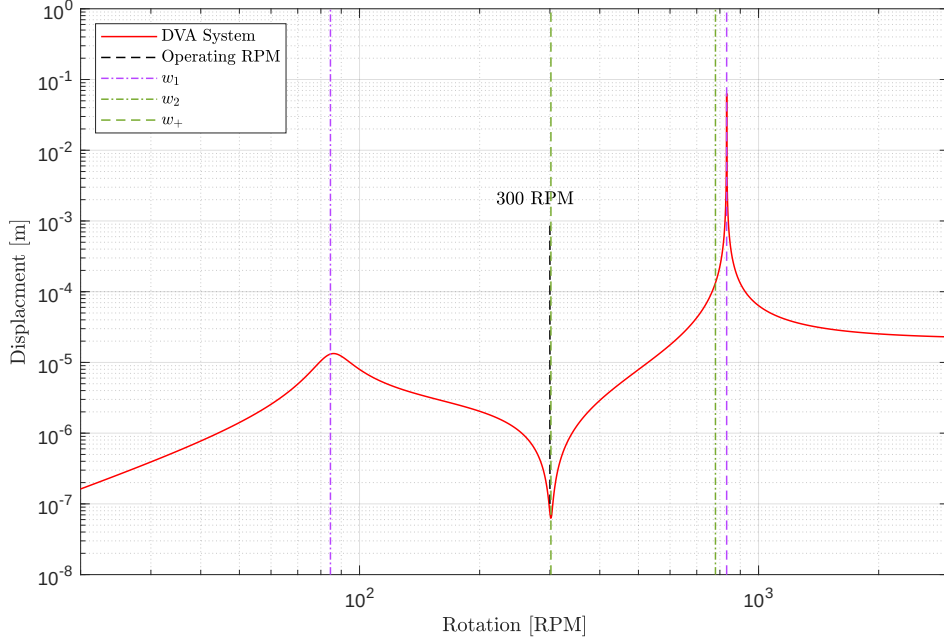


Figure 4: Displacement of the washing machine.

Part 3e

Figure 5 shows the force applied to the floor by the washing machine. *The force applied to the floor at 300 RPM is approximately 3.75×10^{-3} N.*

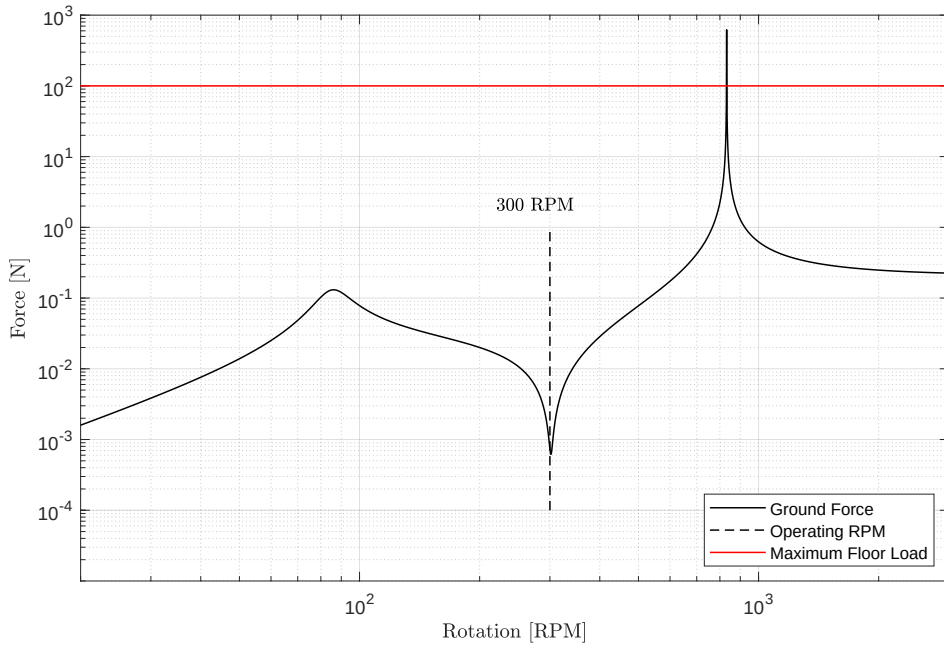


Figure 5: Force applied to the floor.

Problem 4 - Washing Machine Abuse Testing

The Matlab code for this problem is listed in Appendix 4.

Part 4a

Equation 2 is the impulse response for a 1 DOF system.

$$x(t) = \frac{1}{m \cdot w_d} \cdot e^{(-w_o \cdot \zeta \cdot t)} \cdot \sin(w_d \cdot t) \quad (2)$$

where m is the mass of the washing machine, w_d is the damped natural frequency, w_o is the natural frequency, and ζ is the damping ratio.

Figure 1 shows plots of a 1 kg impulse force and the corresponding response of the washing machine.

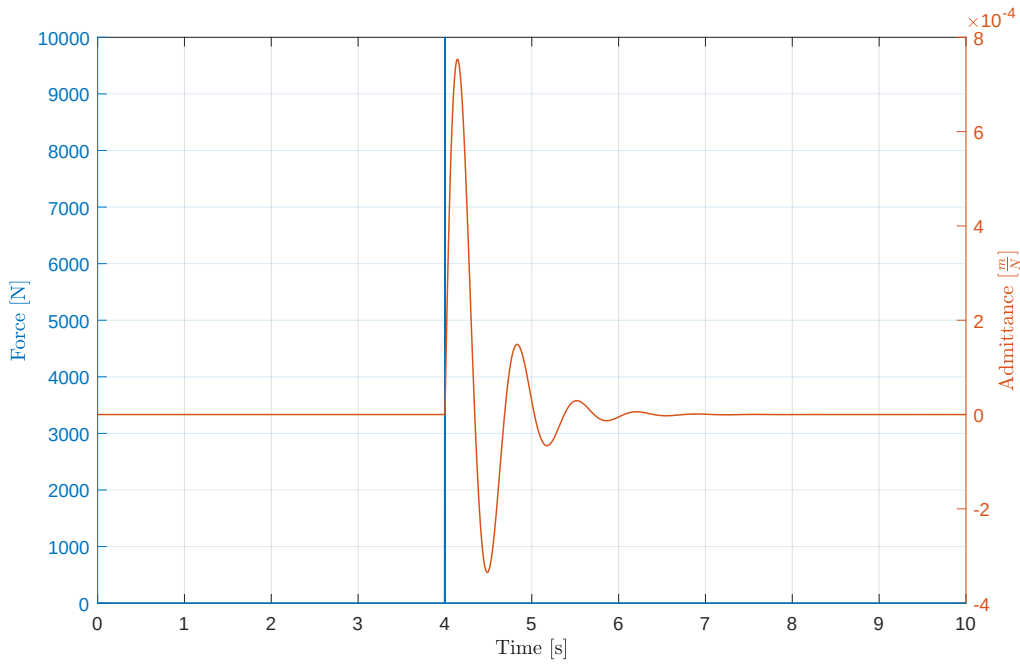


Figure 1: Washing machine impulse response. Here w_o is $9.4 \frac{\text{radians}}{\text{s}}$, w_d is $9.1 \frac{\text{radians}}{\text{s}}$, and ζ is 0.25.

Summary

ph

1 Appendix - Matlab Code for Problem 1

```
1
2
3
4 %% Synopsis
5
6 % Problem 1 – Washing Machine Mount Design – 1 Degree of Freedom (DOF)
7
8
9
10 %% Environment
11
12 close all; clear; clc;
13 % restoredefaultpath;
14
15 addpath( genpath( '../00 Support' ), '-begin' );
16
17 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
18 set( 0, 'DefaultFigureWindowSize', 'normal' );
19 set( 0, 'DefaultLineLineWidth', 0.8 );
20 set( 0, 'DefaultTextInterpreter', 'Latex' );
21
22 format LongG;
23
24 pause( 1 );
25
26
27
28 %% Anonymous Function Definitions
29
30 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
    load_distance;
31
32 h_transmissibility = @( epsilon, r ) sqrt( ( 1 + (2.*epsilon.*r).^2 ) ./ ( ( 1 - r.^2 ).^2 + (
    2.*epsilon.*r ).^2 ) );
33
34 rpm_conversion_to_radians_per_second = 0.10472;
35
36
37
38 %% Washing Machine Information
39
40 machine.mass = 100; % kg
41 machine.rotation_speed = 31.4; % radians\s or 300 rotations per minute (RPM)
42 machine.load_mass = 10; % kg
43 machine.static_displacement = 1e-2; % m
44
45
46
47 %% Load Force Information
48
49 dynamic_force.mass = 1; % kg
50 dynamic_force.radius = 0.5; % m
51
52 force_frequency = 300 * rpm_conversion_to_radians_per_second / ( 2 * pi ); % 5 Hz
53
54 % Maximum force applied to foundation is 100 N.
55
56
57
58 %% Part 1a
59
60 rpm = 0:1:350;
61 angular_velocity = rpm .* rpm_conversion_to_radians_per_second; % radians\s
62
63
64 figure( 'Name', 'Problem 1a – Load Force Versus Angular Velocity' ); ...
65 plot( rpm, h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ) ); hold on;
66 plot( 300, 494, 'Marker', '.', 'MarkerSize', 20 );
67 line( [ 300, 300 ], [ 400 600 ], 'Color', 'r' );
68 text( 305, 494, '493.5 N' ); grid on;
69 xlabel( 'Angular Speed [RPM]' ); ylabel( 'Load Force [N]' );
70 axis( [ -10 355 -5 705 ] );
71
72
73
```

```

74 %% Part 1b
75
76 maximum_allowable_displacement = 1e-2; % m
77 ks = ( machine.load_mass * 9.81) / maximum_allowable_displacement; % 9,810 N\m
78
79
80 total_mass = machine.mass + machine.load_mass; % 110 kg
81 wo = sqrt( ks / total_mass ); % 9.4 radians\s
82 fo = wo / (2*pi); % 1.5 Hz - Natural frequency of the mount.
83
84
85
86 %% Part 1c
87
88 frequency_set = 0.1:0.01:1e3;
89 r = frequency_set ./ fo;
90
91 damping_ratio = logspace( log10(0.001), log10(1), 6 );
92
93
94 figure( 'Name', 'Problem 1c - Transmission Loss' ); ...
95 hold on;
96 %
97 for index = 1:1:numel( damping_ratio )
98     plot( r, 10*log10( h_transmissibility( damping_ratio( index ), r ) ) );
99 end
100 %
101 xlabel( 'Frequency Ratio,  $\frac{f}{f_0}$ ' ); ylabel( 'Transmission Loss [dB]' );
102 legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'NorthEast' );
103 axis( [ 0.1 665 -60 25 ] );
104 grid on; box on;
105 set( gca, 'XScale', 'log' );
106
107
108
109 %% Part 1d
110
111 rpm = 0:1:350; % rotations per minute
112 rpm_conversion_to_radians_per_second = 0.10472; % radians\s
113 angular_velocity = rpm .* rpm_conversion_to_radians_per_second; % radians\s
114
115 frequency_set = angular_velocity / (2*pi);
116 r = frequency_set ./ fo;
117
118
119 figure( 'Name', 'Problem 1d - Applied Force to Foundation' ); ...
120 hold on;
121 %
122 for index = 1:1:numel( damping_ratio )
123     plot( rpm, h_transmissibility( damping_ratio( index ), r ) .* h_force( dynamic_force.mass
124         , angular_velocity, dynamic_force.radius ) );
125 end
126 %
127 line( [ 300, 300 ], [ 2 1e4 ], 'Color', 'k', 'LineStyle', '—' );
128 text( 305, 2, '300 RPM' ); grid on;
129 line( [ 0 350 ], [ 100 100 ], 'Color', 'r', 'LineStyle', '—' );
130 text( 0, 150, '100 N' ); grid on;
131 xlabel( 'Angular Speed [RPM]' ); ylabel( 'Force Applied to Foundation [N]' );
132 legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'South' );
133 axis( [ -10 355 1 2e4 ] );
134 grid on; box on;
135 set( gca, 'YScale', 'log' );
136
137
138 %% Part 1e
139
140 % The best damping ratio is 0.25.
141
142 C = 0.25 * 2*sqrt( ks * total_mass ); % 519.4 kg\s
143
144 % The units of a viscous damping coefficient are Newton-seconds per meter (Ns/m) or kilograms per
145 % second (kg/s).
146
147 m = total_mass;
148 k = [ ks 0 ];
149 dampings = [ C 0 ];

```

```

150     admittance = nDOF_direct_solution( m, k, dampings, frequency_set, 'admittance' );
151
152
153 figure( 'Name', 'Problem 1e - Admittance' ); ...
154     semilogy( rpm, abs( admittance ) ); grid on;
155     xlabel( 'Angular Speed [RPM]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
156     set( gca, 'YScale', 'log' );
157     %
158     line( [ 300, 300 ], [ 7e-6 2e-5 ], 'Color', 'k', 'LineStyle', '—' );
159     text( 305, 2e-5, '300 RPM' ); grid on;
160     axis( [ -10 355 6e-6 2.5e-4 ] );
161
162
163 % temp2 = abs( admittance );
164 %     round( temp2( 301 ), 3, 'significant' )
165
166
167
168 %% Part 1f
169
170 displacement = abs( admittance ) .* h_force( dynamic_force.mass, angular_velocity, dynamic_force.
171     radius ).';
172
173 figure( 'Name', 'Displacement' ); ...
174     plot( rpm, displacement ); grid on;
175     xlabel( 'Angular Speed [RPM]' ); ylabel( 'Displacement [m]' );
176     line( [ 300, 300 ], [ 1e-3 1e-2 ], 'Color', 'k', 'LineStyle', '—' );
177     text( 305, 1e-3, '300 RPM' ); grid on;
178     set( gca, 'YScale', 'log' );
179     axis( [ -10 355 1e-7 2e-2 ] );
180
181
182 temp2 = abs( displacement );
183     round( temp2( 301 ), 3, 'significant' )
184
185
186
187 %% Clean-up
188
189 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
190     monitors = get( 0, 'MonitorPositions' );
191     if ( size( monitors, 1 ) == 1 )
192         autoArrangeFigures( 3, 4, 1 );
193     elseif ( 1 < size( monitors, 1 ) )
194         autoArrangeFigures( 2, 2, 2 );
195     end
196 end
197
198 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
199
200
201
202 %% Reference(s)

```

2 Appendix - Matlab Code for Problem 2

```
1
2
3
4 %% Synopsis
5
6 % Problem 2 — Washing Machine Two-stage Mount Design
7
8
9
10 %% Note(s)
11
12 % A 2 degree-of-freedom (DOF) system will provide isolation performance 1 DOF system.
13
14 % Allowing the top and bottom masses, the washing machine and the raft ,
15 % respectively , to each be attached to ground independently.
16
17 % With the force applied to the washing machine and the displacement of the
18 % washing machine, use FSF( :, 1, 1 ). However, if the force is applied elsewhere ,
19 % than FSF( :, 2, 2, ) must be used.
20
21
22
23 %% Environment
24
25 close all; clear; clc;
26 % restoredefaultpath;
27
28 addpath( genpath( './00 Support' ), '-begin' );
29
30 set( groot, 'DefaultFigurePosition', [ 100 750 750 500 ] ); % x, y, width, height
31
32 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
33 set( 0, 'DefaultFigureWindowStyle', 'normal' );
34 set( 0, 'DefaultLineLineWidth', 0.8 );
35 set( 0, 'DefaultTextInterpreter', 'Latex' );
36
37 format ShortG;
38
39 pause( 1 );
40
41
42
43 %% Define Anonymous Functions
44
45 h_f_to_rpm = @( f ) ( f*2*pi ) / 0.10471;
46
47 h_w_to_rpm = @( w ) h_f_to_rpm( w*2*pi );
48 h_w_to_f = @( w ) w/(2*pi);
49
50 h_rpm_to_f = @( rpm ) ( rpm * 0.10471 ) / ( 2 * pi );
51
52
53 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
54     load_distance;
55
56
57 %% Problem 2a
58
59 % See report .
60
61
62
63 %% Problem 2b — Blocked Frequencies of Resonance
64
65 % The frequency of the forcing function is 5 Hz and the amplitude is 493.5 N.
66 fo = 5; % Hz
67 wo = 2*pi*fo; % 31.4 radians/s
68
69
70 switch ( 1 )
71
72     case 1 % Initial design parameters.
73
74         m1 = 100 + 10; % kg — Total mass of the washing machine and the load.
```



```

75         k1 = 1e2; % N\m
76         c1 = 5;
77
78         m2 = 100; % kg - UPPER LIMIT OF 100 kg
79         k2 = 1e2; % N\m
80         c2 = 5;
81
82         k3 = 1e3; % N\m -> Larger values produce stronger coupling (higher w+).
83
84
85     case 2 % Optimized design parameters.
86
87         m1 = 100 + 10; % kg - Total mass of the washing machine and the load.
88         k1 = 1e2; % N\m
89         c1 = 5;
90
91         m2 = 100; % kg - UPPER LIMIT OF 100 kg
92         k2 = 1e2; % N\m
93         c2 = 5;
94
95         k3 = 1e1; % N\m -> Larger values produce stronger coupling (higher w+).
96
97     end
98
99
100    w1 = sqrt( ( k1 + k3 ) / m1 ); % 3.2 radians\s; washing machine
101        f1 = w1 / (2*pi); % 0.5 Hz
102        rpm1 = f1*(2*pi)/0.10471; % 30.1 RPM
103
104    w2 = sqrt( ( k2 + k3 ) / m2 ); % 3.3 radians\s; raft
105        f2 = w2 / (2*pi); % 0.53 Hz
106        rpm2 = f2*(2*pi)/0.10471; % 31.7 RPM
107
108    % Note: If k3 is set to zero, then these frequencies represent the
109    % resonance frequencies for each mass on their own.
110
111
112
113 %% Problem 2b - Coupled Frequencies
114
115    mu_4 = ( k3^2 / (m1*m2) ); % 90.9 (radians\s)^4
116    mu = ( mu_4 ) ^0.25; % 3.1 radians\s
117
118    w(1) = 0.5*( ( w1^2 + w2^2 ) + sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
119    w(2) = 0.5*( ( w1^2 + w2^2 ) - sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
120    w = sqrt( w );
121
122
123    w_minus = w(2); % 0.97 radians\s (lower than w1, 1, and w2, 1.05 radians\s)
124    f_minus = w(2)/(2*pi); % 0.1545 Hz
125    rpm_minus = h_f_to_rpm( f_minus ); % 9.3 RPM
126    %
127    w_plus = w(1); % 4.5 radians\s (higher than w1, 1, and w2, 1.05 radians\s)
128    f_plus = w(1)/(2*pi); % 0.71 Hz
129    rpm_plus = h_f_to_rpm( f_plus ); % 42.8 RPM
130    %
131    % Note(s):
132    %
133    % The blocked frequencies, w1 and w2, are always between these two frequencies.
134
135    clear w mu_4;
136
137
138
139 %% Problem 2b - Mode Shapes
140
141    phi_plus = [ ...
142        1; ...
143        -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
144        ];
145    %
146    % 1
147    % -1.10
148
149    phi_minus = [ ...
150        1; ...
151        (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
152        ];

```

```

153 %
154 % 1
155 % 0.99
156
157
158 % Note(s):
159 %
160 % When m1 = m2 and k1 = k3,
161 %
162 % a.) At w_minus, m1 and m2 move the same amount and are in-phase.
163 % b.) At w_plus, m1 and m2 move the same amount, but are out-of-phase.
164
165 % In general, for a 2 DOF system there will be 2 modes (in-phase and out-of-phase).
166
167 % With different initial conditions (no forcing), the behaviour is a weighted sum of the two
    modes.
168
169
170
171 %% Problem 2c
172
173 masses = [ m1 m2 ];
174 stiffnesses = [ k1 k3 k2 ];
175 dampings = [ c1 0 c2 ];
176
177
178 rpm = 0:0.1:500; % rotations per minute
179 rpm_conversion_to_radians_per_second = 0.10472;
180 angular_velocity = rpm .* rpm_conversion_to_radians_per_second; % radians\s
181 frequencies = angular_velocity / (2*pi); % Hz
182
183 FRF = nDOF_direct_solution( masses, stiffnesses, dampings, frequencies, 'admittance' );
184
185
186 admittance = zeros( numel( frequencies ), 1 );
187
188 for index = 1:1:numel( frequencies )
189     temp = diag( squeeze( FRF( index, 1, 1 ) ) ); % Check indexing to ensure force on washing
        machine.
190     admittance( index ) = abs( temp );
191 end
192 %
193 clear temp temp2;
194
195 washing_machine.admittance = admittance;
196
197
198 admittance = zeros( numel( frequencies ), 1 );
199
200 for index = 1:1:numel( frequencies )
201     temp = diag( squeeze( FRF( index, 1, 2 ) ) ); % Check indexing to ensure force on washing
        machine.
202     admittance( index ) = abs( temp );
203 end
204 %
205 clear temp temp2;
206
207 raft.admittance = admittance;
208
209
210 figure( 'Name', 'Admittance' ); ...
211 h1 = loglog( rpm, washing_machine.admittance, 'Color', 'r', 'LineWidth', 0.8 ); hold on;
212 h2 = loglog( rpm, raft.admittance, 'Color', 'b', 'LineWidth', 0.8 );
213 h3 = line( [ 300 300 ], [ 1e-6 1e-4 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
214 text( 220, 2e-4, '300 RPM' );
215 %
216 h4 = line( [ rpm_minus rpm_minus ], [ 1e-8 1e1 ], 'Color', [ 0.72, 0.27, 1.00 ], 'LineStyle',
    '-.' );
217 %
218 h5 = line( [ rpm1 rpm1 ], [ 1e-8 1e1 ], 'Color', [ 0.47, 0.67, 0.19 ], 'LineStyle', '-.' );
219 h6 = line( [ rpm2 rpm2 ], [ 1e-8 1e1 ], 'Color', [ 0.47, 0.67, 0.19 ], 'LineStyle', '-' );
220 %
221 h7 = line( [ rpm_plus rpm_plus ], [ 1e-8 1e1 ], 'Color', [ 0.72, 0.27, 1.00 ], 'LineStyle',
    '-', '—' );
222 %
223 legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Washing Machine', 'Raft', 'Load Frequency', 'w-', 'w1', '
    w2', 'w+', 'Location', 'NorthEast' );
224 %

```

```

225     xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
226     axis( [ 6 400 1e-8 1e1 ] );
227
228
229     round( washing_machine.admittance( 3001 ), 3, 'significant' )
230
231
232
233 %% Problem 2d
234
235     dynamic_force.mass = 1; % kg
236     dynamic_force.radius = 0.5; % m
237
238
239     figure( 'Name', 'Displacement' ); ...
240         h1 = loglog( rpm, washing_machine.admittance.*h_force( dynamic_force.mass, angular_velocity,
241             dynamic_force.radius ).', 'Color', 'r' ); hold on;
241         h2 = loglog( rpm, raft.admittance.*h_force( dynamic_force.mass, angular_velocity,
242             dynamic_force.radius ).', 'Color', 'b' );
242         h3 = line( [ 300 300 ], [ 1e-5 1e-1 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
243         text( 220, 0.21, '300 RPM' );
244         legend( [ h1 h2 h3 ], 'Washing Machine', 'Raft', 'Load Frequency', 'Location', 'South' );
245         xlabel( 'Rotation [RPM]' ); ylabel( 'Displacement [m]' );
246         axis( [ 6 400 1e-6 1 ] );
247
248
249     temp = washing_machine.admittance.*h_force( dynamic_force.mass, angular_velocity, dynamic_force.
250         radius ).';
251     round( temp( 3001 ), 3, 'significant' )
252
253     % temp = raft.admittance.*h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius )
254     % round( temp( 3001 ), 3, 'significant' )
255
256
257 %% Problem 2e
258
259 % Displacement of the raft multiplied by the K2 spring constant.
260
261     dynamic_force.mass = 1; % kg
262     dynamic_force.radius = 0.5; % m
263
264     h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
265         load_distance;
266
267     temp = h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).';
268
269     figure( 'Name', 'Force Applied to Ground by Raft' ); ...
270         h1 = loglog( rpm, raft.admittance.*temp*k2, 'Color', 'b' ); hold on;
271         h2 = line( [ 300 300 ], [ 2e-3 1e-1 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
272         h3 = line( [ 6 400 ], [ 100 100 ], 'Color', 'r' );
273         text( 220, 2e-1, '300 RPM' );
274         legend( [ h1 h2 h3 ], 'Raft Force', 'Load Frequency', 'Maximum Floor Load', 'Location', '
275             South' );
275         xlabel( 'Rotation [RPM]' ); ylabel( 'Force Applied to Ground [N]' );
276         axis( [ 6 400 1e-3 1e3 ] );
277
278
279     temp = raft.admittance.*h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).'*
280         k2;
281     round( temp( 3001 ), 3, 'significant' )
282
283
284 %% Problem 2f
285
286 % return
287
288 %% Problem 2g
289
290 % return
291
292 %% Clean-up
293
294 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
295     monitors = get( 0, 'MonitorPositions' );

```

```

296         if ( size( monitors, 1 ) == 1 )
297             autoArrangeFigures( 3, 4, 1 );
298         elseif ( 1 < size( monitors, 1 ) )
299             autoArrangeFigures( 2, 2, 2 );
300         end
301     end
302
303     fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );
304
305
306
307     %% Reference(s)
308
309
310
311     %% Comment(s)
312
313
314
315     %% Regions of Interest
316
317     % 5 regions of interest:
318
319     % 1.) Low frequency:  $w < w_{\text{minus}}$ :
320     %
321     % Stiffness controlled; two masses moving together; strong coupling ( $k_3$  high).
322
323
324     % 2.) Force at a resonance frequency:  $w = w_{\text{minus}}$  (lowest resonance frequency):
325     %
326     % At lowest resonance frequency; at  $w_{\text{minus}}$  mode, masses moving in same direction (in-phase)
327     % with same high amplitude
328
329     % 3.) Forcing frequency between  $w_{\text{minus}}$  and  $w_2$ 
330
331
332     % 4.) Forcing frequency  $w = w_2$ 
333
334
335     % 5.) Forcing frequency  $w < w < w_+$ 
336     %
337     % No a special frequency.
338
339
340     % 6.) Forcing frequency  $w = w_+$ 
341     %
342     % At highest resonance frequency; at  $w_{\text{plus}}$  mode, masses moving in opposite directions (out-of-
343     % phase) with same high amplitude
344
345     % 7.) Forcing frequency  $w_+ < w$ 
346     %
347     % Mass controlled region; mass 1 (washing machine) moves more than mass 2 (raft);

```

3 Appendix - Matlab Code for Problem 3

```
1
2
3 % To do:
4
5 % remove comment statements
6
7 % verify indices for required values
8
9
10
11 %% Synopsis
12
13 % Problem 3 – Washing Machine Dynamic Vibration Absorber (DVA) Design
14
15
16
17 %% Note(s)
18
19 % At 300 RPM, the load frequency, this system will isolated vibration more
20 % than the 2 DOF and 1 DOF systems.
21
22 % The DVA behaves like a Helmholtz resonator, working as a mechanical notch
23 % filter at a particular frequency.
24
25 % DVAs work well in systems that have one dominate mode. A good
26 % example of this is building sway and its compensation. Based on the
27 % design and construction of a building, it will typically have a single,
28 % dominate mode.
29
30 % DVA is useful if you can not get away from the resonance frequency; can
31 % not tune a 2 DOF system when the resonance frequency is away from the
32 % forcing frequency.
33
34 % DVA introduces two resonances for the original resonance. These new
35 % resonances will produce greater motion at the new resonance frequencies.
36
37
38
39 %% Environment
40
41 close all; clear; clc;
42 % restoredefaultpath;
43
44 addpath( genpath( './00 Support' ), '-begin' );
45
46 set( groot, 'DefaultFigurePosition', [ 100 750 750 500 ] ); % x, y, width, height
47
48 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
49 set( 0, 'DefaultFigureWindowSize', 'normal' );
50 set( 0, 'DefaultLineLineWidth', 0.8 );
51 set( 0, 'DefaultTextInterpreter', 'Latex' );
52
53 format ShortG;
54
55 pause( 1 );
56
57
58
59 %% Define Anonymous Functions
60
61 h_rpm_to_f = @( rpm ) ( rpm * 0.10471 ) / ( 2 * pi );
62 h_rpm_to_w = @( rpm ) ( rpm * 0.10471 );
63
64 h_f_to_rpm = @( f ) ( f*2*pi ) / 0.10471;
65
66 h_w_to_f = @( w ) w/(2*pi);
67 h_w_to_rpm = @( w ) h_f_to_rpm( w*2*pi );
68
69
70 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
    load_distance;
71
72
73
74 %% Washing Machine Information
```

```

75
76 machine.mass = 100; % kg
77 machine.rotation_speed = 31.4; % radians\s or 300 rotations per minute (RPM)
78 machine.load_mass = 10; % kg
79 machine.static_displacement = 1e-2; % m
80
81
82
83 %% Load Force Information
84
85 dynamic_force.mass = 1; % kg
86 dynamic_force.radius = 0.5; % m
87
88 % Maximum force applied to foundation is 100 N.
89
90
91
92 %% Dynamic Vibration Absorber (DVA)
93
94 maximum_allowable_displacement = 1e-2; % m
95 ks = ( machine.load_mass * 9.81) / maximum_allowable_displacement; % 9,810 N\m
96
97
98 m = 110; % kg; washing machine mass and load mass
99 k = ks; % 9,810 N\m
100
101 wo = sqrt( k / m); % 9.4 radians\s
102 fo = wo/(2*pi); % 1.5 Hz
103
104
105 m_dva = 15; % kg
106 k_dva = 1e5; % N\m
107
108 wo_dva = sqrt( k_dva / m_dva ); % 81.7 radians\s
109 fo_dva = wo_dva/(2*pi); % 13.0 Hz
110
111
112
113 %% Blocked Frequencies (Equation 9.1 of the Notes)
114
115 m1 = m_dva;
116 k1 = 0; % Spring above DVA mass; does not exist; set to zero.
117
118 m2 = m;
119 k2 = k;
120
121 k3 = k_dva;
122
123
124 w1 = sqrt( ( k1 + k3 ) / m1 ); % 81.7 radians\s; smaller than the system resonant frequency.
125 f1 = w1/(2*pi); % 13 Hz
126
127 w2 = sqrt( ( k2 + k3 ) / m2 ); % 31.6 radians\s; greater than the system resonant frequency.
128 f2 = w2/(2*pi); % 5.0 Hz
129
130
131
132 %% Coupled Frequencies (Equation 9.15 of the Notes)
133
134 mu_4 = ( k3^2 / (m1*m2) ); % 6.1 (kg\s)^4
135 mu = ( mu_4 )^0.25; % 49.1 kg\s
136
137 w(1) = 0.5*( ( w1^2 + w2^2 ) + sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
138 w(2) = 0.5*( ( w1^2 + w2^2 ) - sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
139 w = sqrt( w );
140
141
142 w_minus = w(2); % 8.9 radians\s (lower than 31.6 and 81.7 radians\s)
143 f_minus = w(2)/(2*pi); % 1.4 Hz
144 %
145 w_plus = w(1); % 87.1 radians\s (greater than 31.6 and 81.7 radians\s)
146 f_plus = w(1)/(2*pi); % 13.9 Hz
147
148
149 % The blocked frequencies are always between the lower and higher coupled frequencies.
150
151
152

```

```

153 %% Mode Shapes (Equations 9.18 and 9.19 of the Notes)
154
155 phi_plus = [ ...
156     1; ...
157     -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
158 ];
159 %
160 % 1
161 % -0.14
162
163
164 phi_minus = [ ...
165     1; ...
166     (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
167 ];
168 %
169 % 1
170 % 0.99
171
172
173
174 %% Admittance Using Equations from DVA Lecture
175
176 MAXIMUM_RPM = 3e3;
177
178 F0 = 1;
179
180 w = 0:0.01:h_rpm_to_w( MAXIMUM_RPM );
181 f = w/(2*pi);
182 rpm = h_f_to_rpm( f );
183
184
185 m = [ 15 110 ];
186 k = [ 0 k_dva.*(1+0.1*i) k2.*(1+0.2*i) ];
187
188
189 admittance_modified = F0 ./ ( m(1)*m(2) ) * k(2) ./ (( w.^2 - w_plus^2 ) .* ( w.^2 - w_minus^2 ));
190 admittance_original = F0 ./ m(2) * 1 ./ ( w2^2 - w.^2 );
191
192
193 figure( 'Name', 'Admittance - Equations from DVA Lecture' ); ...
194 h1 = loglog( rpm, abs( admittance_original ) ); hold on;
195 h2 = loglog( rpm, abs( admittance_modified ) );
196 h3 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '—' );
197 text( 220, 2e-2, '300 RPM' );
198 h4 = line( [ h_f_to_rpm( f_minus ) h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
199     0.27, 1.00 ], 'LineStyle', '—.' );
200
201 h5 = line( [ h_f_to_rpm( f1 ) h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
202     ], 'LineStyle', '—.' );
203 h6 = line( [ h_f_to_rpm( f2 ) h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
204     ], 'LineStyle', '—' );
205
206 h7 = line( [ h_f_to_rpm( f_plus ) h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
207     0.27, 1.00 ], 'LineStyle', '—' ); grid on;
208
209 axis( [ 5 3e3 1e-7 1 ] );
210 legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Original System', 'DVA System', 'Operating RPM', ...
211     '$w_{-}$', '$w_{1}$', ...
212     '$w_{2}$', '$w_{+}$', ...
213     'Location', 'NorthWest', 'Interpreter', 'Latex' );
214 xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance [$\frac{m}{N}$]' );
215
216 return
217
218 %% Admittance Using nDOF Function
219
220 % Damping can be added by using a dampings vector or appending a complex
221 % value to the respective spring stiffness. THESE ARE NOT INTERCHANGABLE.
222
223 rpm = 0:0.1:MAXIMUM_RPM;
224 f = h_rpm_to_f( rpm );
225 w = f ./ (2*pi);
226
227 FRF = nDOF_direct_solution( m, k, [ 0 0 0 ], f, 'admittance' ); % Symmetric matrix.
228 %
229 squeeze( FRF( numel( f ), :, : ) );

```

```

227 %
228 % (1, 1) – Force on M1 and its associated displacement.
229 % (2, 2) – Force on M2 and its associated displacement.
230 %
231 % (1, 2) – Force on M1 and the displacement of M2.
232 % (2, 1) – Force on M2 and the displacement on M1.
233 %
234 % These are interchangeable; the same result.
235
236
237 figure( 'Name', 'Admittance – nDOF Calculated' ); ...
238 h1 = loglog( h_f_to_rpm( f ), abs( FRF( :, 1, 1 ) ), 'Color', 'r' ); hold on;
239 h2 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '—' );
240 text( 220, 2e-2, '300 RPM' );
241 h3 = line( [ h_f_to_rpm( f_minus ) h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
242 0.27, 1.00 ], 'LineStyle', '—.' );
243
244 h4 = line( [ h_f_to_rpm( f1 ) h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
245 ], 'LineStyle', '—.' );
246 h5 = line( [ h_f_to_rpm( f2 ) h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
247 ], 'LineStyle', '—.' );
248
249 h6 = line( [ h_f_to_rpm( f_plus ) h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
250 0.27, 1.00 ], 'LineStyle', '—.' ); grid on;
251
252 axis( [ 2e1 MAXIMUM_RPM 1e-7 1 ] );
253 legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
254 '$w_{-}$', '$w_{1}$', ...
255 '$w_{2}$', '$w_{+}$', ...
256 'Location', 'NorthWest', 'Interpreter', 'Latex' );
257 xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
258
259 % temp2 = abs( FRF( :, 1, 1 ) );
260 % round( temp2( 3001 ), 3, 'significant' )
261
262 %% Displacement
263
264 dynamic_force.mass = 1; % kg
265 dynamic_force.radius = 0.5; % m
266
267 figure( 'Name', 'Displacement' ); ...
268 h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius
269 ).', 'Color', 'r' ); hold on;
270 h2 = line( [ 300 300 ], [ 1e-7 1e-3 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
271 text( 220, 2e-3, '300 RPM' );
272 h4 = line( [ h_f_to_rpm( f_minus ) h_f_to_rpm( f_minus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
273 0.27, 1.00 ], 'LineStyle', '—.' );
274
275 h5 = line( [ h_f_to_rpm( f1 ) h_f_to_rpm( f1 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
276 ], 'LineStyle', '—.' );
277 h6 = line( [ h_f_to_rpm( f2 ) h_f_to_rpm( f2 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
278 ], 'LineStyle', '—.' );
279
280 h7 = line( [ h_f_to_rpm( f_plus ) h_f_to_rpm( f_plus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
281 0.27, 1.00 ], 'LineStyle', '—.' ); grid on;
282
283 legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
284 '$w_{-}$', '$w_{1}$', ...
285 '$w_{2}$', '$w_{+}$', ...
286 'Location', 'NorthWest', 'Interpreter', 'Latex' );
287
288 xlabel( 'Rotation [RPM]' ); ylabel( 'Displacement [m]' );
289 axis( [ 2e1 MAXIMUM_RPM 1e-8 1 ] );
290
291 % temp2 = abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius ).';
292 % round( temp2( 3001 ), 3, 'significant' )
293
294 %% Ground Force
295
296 dynamic_force.mass = 1; % kg
297 dynamic_force.radius = 0.5; % m

```



```

296
297
298 figure( 'Name', 'Ground Force' ); ...
299 h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius
300             ).'*k2, 'Color', 'k' ); hold on;
301 h2 = line( [ 300 300 ], [ 1e-4 1e-0 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
302 text( 220, 2e-0, '300 RPM' );
303 h3 = line( [ 6 MAXIMUM_RPM ], [ 100 100 ], 'Color', 'r' );
304 legend( [ h1 h2 h3 ], 'Ground Force', 'Operating RPM', 'Maximum Floor Load', 'Location',
305         'SouthEast' );
306 xlabel( 'Rotation [RPM]' ); ylabel( ' Force [N]' );
307 axis( [ 2e1 MAXIMUM_RPM 1e-5 1e3 ] );
308
309 temp2 = abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius ).'*k2;
310 % round( temp2( 3001 ), 3, 'significant' )
311
312
313 %% Clean-up
314
315 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
316     monitors = get( 0, 'MonitorPositions' );
317     if ( size( monitors, 1 ) == 1 )
318         autoArrangeFigures( 3, 4, 1 );
319     elseif ( 1 < size( monitors, 1 ) )
320         autoArrangeFigures( 2, 2, 2 );
321     end
322 end
323
324 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
325
326
327
328 %% Reference(s)

```

4 Appendix - Matlab Code for Problem 4

```
1
2
3
4 %% Synopsis
5
6 % Problem 4 – Washing Machine Abuse Testing
7
8
9
10 %% Environment
11
12 close all; clear; clc;
13 % restoredefaultpath;
14
15 addpath( genpath( './00 Support' ), '-begin' );
16
17 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
18 set( 0, 'DefaultFigureWindowSize', 'normal' );
19 set( 0, 'DefaultLineLineWidth', 0.8 );
20 set( 0, 'DefaultTextInterpreter', 'Latex' );
21
22 format ShortG;
23
24 pause( 1 );
25
26
27
28 %% Define Anonymous Functions
29
30 h_unit_impulse = @( washing_machine_mass, wd, wo, epsilon, t ) ( 1./( washing_machine_mass.*wd)
31 ) .* exp( -wo.*epsilon.*t ) .* sin( wd.*t ); % mvo = 1 kg\ (m s)
32
33
34 %% Impulse Signal
35
36 time_step = 1e-4; % s
37 net_time = 10; % s
38 time_indices = 0:time_step:( net_time - time_step );
39
40 impulse = zeros( size( time_indices ) );
41 impulse( 4/time_step ) = 1./time_step;
42 brick_impulse = 5 .* impulse;
43
44
45
46 %% 1 DOF System Impulse Function
47
48 ks = 9810; % N\m
49 wo = sqrt( ks / 110 ); % 9.4 radians\s
50 fo = wo / (2*pi); % 1.5 Hz
51
52 epsilon = 0.25;
53 C = epsilon * 2*sqrt( ks * 110 ); % 519.4 kg\s
54
55 wd = wo * sqrt( 1 - epsilon^2 ); % 9.1 radians\s
56 fd = wd / (2*pi); % 1.46 Hz
57
58
59
60 %% 1 DOF System Impulse Response
61
62 impulse_response = h_unit_impulse( 100, wd, wo, epsilon, time_indices );
63
64 impulse_response_at_4 = conv( impulse, impulse_response ) * time_step;
65 time_indices_at_4 = ( 0:1:( numel( impulse_response_at_4 ) - 1 ) ) .* time_step;
66 %
67 figure( 'Name', '1 DOF System Impulse Response' ); ...
68 yyaxis left; ...
69 plot( time_indices, impulse ); grid on;
70 ylabel( 'Force [N]' );
71 ylim( 'auto' );
72 yyaxis right; ...
73 plot( time_indices_at_4, impulse_response_at_4 ); hold on;
74 ylabel( 'Admittance [ $\frac{m}{N}$  s]' );
```

```

75     ylim( 'auto' );
76     xlabel( 'Time [s]' );
77     xlim( [ 0 10 ] );
78
79     return
80
81     response_to_brick_perturbation = conv( brick_impulse, impulse_response ) * time_step;
82     time_indices_perturbation = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
        time_step;
83 %
84 figure( 'Name', '1 DOF Impulse Response' ); ... % Figure 3
85     plot( time_indices_perturbation, response_to_brick_perturbation ); grid on;
86     xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
87     % axis( [ -0.1 5 -1.5e-3 1.5e-3 ] );
88
89     return
90
91 %% Sinusoidal Impulse Response
92
93     sinusoid_input = 493.5 * sin( 2 * pi * 5 * time_indices );
94 % sinusoid_input = 1 * sin( 2 * pi * 5 * time_indices );
95 % sinusoid_input = 493.5 * sin( 2 * pi * 1 * time_indices );
96     sinusoid_input( 1:1:( 1 / time_step ) ) = 0;
97     sinusoid_input( ( 8 / time_step ):1:( 10 / time_step ) ) = 0;
98 %
99     figure( 'Name', 'Sinusoidal Input' ); ... % Figure 4
100     plot( time_indices, sinusoid_input ); grid on;
101     xlabel( 'Time [s]' ); ylabel( 'Force [N]' );
102     % xlim( [ 0 2.5 ] );
103     ylim( [ -600 600 ] );
104
105
106     response_to_sinusoidal_forcing = conv( sinusoid_input, impulse_response ) * time_step;
107     time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_sinusoidal_forcing ) - 1 ) ) .*
        time_step;
108 %
109     figure( 'Name', '1 DOF Sinusoidal Forcing Response' ); ... % Figure 5
110     plot( time_indices_sinusoidal_forcing, response_to_sinusoidal_forcing ); grid on;
111     xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
112     % xlim( [ 0 2.5 ] );
113
114
115
116 %% Combined Response
117
118     joint_signal = sinusoid_input + brick_impulse;
119 % %
120 % figure( 'Name', 'Combined Signal' ); ... % Figure 6
121 %     plot( time_indices, joint_signal ); hold on;
122 %     plot( time_indices, sinusoid_input, 'LineStyle', '--' );
123 %     plot( time_indices, impulse, 'LineStyle', '--' );
124 %     xlabel( 'Time [s]' ); ylabel( 'Force [N]' );
125 %     % xlim( [ 0 2.5 ] );
126 %     % ylim( [ -600 600 ] );
127
128
129     response_to_combined_forcing = conv( impulse_response, joint_signal ) * time_step;
130     time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
        time_step;
131 %
132     figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
133     plot( time_indices_sinusoidal_forcing, response_to_combined_forcing ); grid on;
134     xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
135     % xlim( [ 0 2.5 ] );
136
137
138
139 %% Acceleration
140
141     velocity = movingslope( response_to_combined_forcing );
142     velocity2 = gradient( response_to_combined_forcing );
143 %
144     figure( 'Name', 'Velocity' ); ...
145     plot( time_indices_sinusoidal_forcing, velocity ); hold on;
146     plot( time_indices_sinusoidal_forcing, velocity2, 'LineStyle', '--' ); grid on;
147     xlabel( 'Time [s]' ); ylabel( 'Velocity [ $\frac{m}{s}$ ]' );
148     % xlim( [ 0 2.5 ] );
149

```

```

150
151 acceleration = movingslope( velocity );
152 acceleration2 = gradient( velocity2 );
153 %
154 figure( 'Name', 'Acceleration' ); ...
155     plot( time_indices_sinusoidal_forcing, acceleration ); hold on;
156     plot( time_indices_sinusoidal_forcing, acceleration2, 'LineStyle', '—' ); grid on;
157     xlabel( 'Time [s]' ); ylabel( 'Acceleration [ $\frac{m}{s^2}$ ]' );
158     % xlim( [ 0 2.5 ] );
159
160
161
162 %% Verify
163
164 close all;
165
166 t = time_indices_sinusoidal_forcing;
167 x = sin( 2 * pi * 5 * t );
168
169 time_step_verify = t(2) - t(1);
170
171 figure( ); ...
172     plot( t, x ); hold on;
173     % plot( t(1:end-1), diff( x ) / time_step_verify );
174     plot( t, gradient( x ) / time_step_verify );
175     grid on;
176     % axis( [ 0 1 -1.2 1.2 ] );
177
178
179
180 %% Plot Difference
181
182 % figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
183 %     plot( time_indices_sinusoidal_forcing, response_to_combined_forcing -
184 %         response_to_sinusoidal_forcing ); grid on;
185 %     xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
186 %     % xlim( [ 0 2.5 ] );
187
188
189 %% Clean-up
190
191 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
192     monitors = get( 0, 'MonitorPositions' );
193     if ( size( monitors, 1 ) == 1 )
194         autoArrangeFigures( 3, 4, 1 );
195     elseif ( 1 < size( monitors, 1 ) )
196         autoArrangeFigures( 2, 2, 2 );
197     end
198 end
199
200 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
201
202
203
204 %% Reference(s)

```